

Documentação Explicativa Arquitetural

Projeto: Extrator ClickUp PRO V9 (Aderência Padrão Digital PE)

Este documento visa detalhar, de forma técnica e resumida, o papel individual dos cinco scripts que compõem o ecossistema do Motor de Extração de back-ups. A divisão foi projetada focando em **Alta Coesão** e **Baixo Acoplamento**.

1. O Motor Visual Dinâmico (gui.py)

Este é o portal de entrada do usuário. Desenvolvido sobre o moderno **customtkinter**, entrega uma autêntica experiência UI Corporativa baseada na paleta institucional. Sua principal virtude de código é o paralelismo (**Threading**): ele despacha a requisição de inicialização em segundo plano, evitando travamentos na tela, enquanto injeta magicamente a leitura do terminal direto no quadro verde-neon da Interface via *StdoutRedirector*.

```
def run_extraction_logic(is_pilot, ws_id, ...):
    # Emula chamadas reais de Terminal
    sys.argv = ['main.py', '--mode', mode, '--workspace', ws_id]
    import core_main
    core_main.main()
```

2. O Cérebro do Negócio (main.py)

Considerado a engrenagem principal. Ele coordena o looping vital entre: Workspace -> Espaços -> Pastas -> Listas -> Tarefas. Seu maior e mais refinado trunfo no código é o mecanismo de **Força Bruta e Pré-Fetch de Pais**. Devido a limitações do próprio ClickUp, tarefas-pai podem ficar 'invisíveis' nos filtros, então o script identifica se uma filha está órfã e reconstrói o pai recursivamente na memória, garantindo estruturação perfeita das hierarquias físicas.

```
if parent_id and tasks_dict and parent_id in tasks_dict:
    p_task = tasks_dict[parent_id]
    # O Resumo do pai é forçado cirurgicamente para evitar perda!
    p_path = writer.create_hierarchy(task_id=parent_id, ...)
    writer.save_human_readable_pdf(p_path, p_task, [])
```

3. A Casca de Conexão Segura (clickup_client.py)

Responsável 100% pelos envios HTTP para os datacenters da nuvem ClickUp. Não possui lógica de geração visual, é um microserviço puramente de rede. O código é rigorosamente protegido contra problemas de **Rate Limits (Erro 429)** e Perdas de Conexão intermitente da API de extração.

```
def _get(self, endpoint, params):
    for attempt in range(retries=3):
        res = requests.get(url, headers=self.headers)
        if res.status_code == 429: # Trata Engarrafamento ClickUp
            time.sleep(1.5)
        return res.json()
```

4. O Construtor Físico & Sanitizador PDF (data_writer.py)

O pedreiro do sistema. Responsável por traduzir os dados da nuvem para as Pastas em C: e materializar toda a vida do funcionário em PDFs através da robusta biblioteca **ReportLab**. Seu código mais importante previne um colapso fatal da geração de PDFs quando usuários inserem caracteres Unicode bizarros/invisíveis nas descrições originais (Lógica de Sanitização por String Encoding).

```
def clean_text_for_pdf(text):
    text = str(text).replace('\u2028', ' ')
    # Substitui caracteres explosivos que derrubariam o relatório
    return ''.join(c for c in text if ord(c) < 256)
```

5. O Recuperador de Mídias Silencioso (attachment_downloader.py)

Lê os metadados de assinaturas de URLs. Faz download binário sob forte sistema tolerante a falhas (try-catch), assegurando que o arquivo nunca corrompa. O módulo é recursivo e visita os logs de comentário também.

--- Documento Gerado pela Automação de Infraestrutura V9 ---